## CSCI 5280 – Software-Defined Networking

# Lab 7

## Natural-Language Network Controller: Intent to Configuration

University of Colorado Boulder
Department of Computer Science
Network Engineering

**Professor Levi Perigo, Ph.D.**

## Lab Summary

In Lab 6 you taught a system to read a network and explain what it sees. In this lab you teach it to listen to a human and do what they say.

Intent-Based Networking is one of the most actively developed concepts in modern network engineering. Cisco, Juniper, and virtually every major vendor have products that claim to implement it. The core idea is deceptively simple: an operator expresses what they want the network to do in plain language, and the system figures out how to make it happen. The hard part is the translation — from natural language, which is ambiguous and context-dependent, to structured configuration, which must be precise and unambiguous.

This lab builds that translation engine. You will create a Python pipeline that accepts a natural-language network policy statement, sends it to an LLM to be translated into a structured configuration object, validates that the output is actionable, and applies it to a live Mininet network via REST. By the end of this lab you will have a working natural-language controller: type what you want, and the network does it.

Reference: Chou, AI Networking Cookbook, Chapters 4–5. Leivadeas & Falkner — A Survey on Intent-Based Networking (Canvas). Cisco — AI-Native Networking white paper (Canvas).

> **API Access:** University ChatGPT Pro access is available to all enrolled students — see Canvas for setup instructions. You may also use Claude or Gemini. Use economy-tier models for development: GPT-4o-mini, Claude Haiku, or Gemini Flash. Disclose your AI tool use in your lab report.

## Objective 1 – Define the Intent Schema and Translation Pipeline

Before writing any LLM prompt, you need to decide what a translated network intent looks like as structured data. That schema is the contract between your translator and your network applier — if the schema is wrong, nothing downstream works. Design it first.

**1.** Design a JSON schema that represents a translated network policy. Your schema must be expressive enough to represent at least the following policy types: **[15 points]**

- Block traffic between two hosts or subnets
- Allow specific traffic (by protocol or port) between two hosts
- Rate-limit traffic from a source to a destination
- Prioritize traffic of a specific type

Your schema should include at minimum: action type, source, destination, protocol, port (where applicable), and any action-specific parameters. Document your schema with a brief explanation of each field. There is no single correct answer — design it to be both expressive and parseable by your downstream code.

**2.** Set up your Mininet environment for this lab: a single switch connected to your chosen SDN controller with four hosts (h1–h4), OpenFlow 1.3, easy-to-read MAC addresses. Verify the topology is running and all hosts can communicate. Provide a screenshot.

I used Ryu on the controller VM and a single OpenFlow 1.3 switch with four hosts. The controller is 10.224.78.132, and Mininet connects to it over the internal LAN on port 6633. Tailscale is just for SSH access.

The hosts are h1-h4 with addresses 10.0.0.1-10.0.0.4/24 and MAC addresses 00:00:00:00:00:01-00:00:00:00:00:04. I ran pingall and got 0% dropped, with all 12 host-to-host checks received.

Mininet topology and pingall. Actual terminal screenshot over SSH.

```
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> nodes
available nodes are:
c0 h1 h2 h3 h4 s1
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0
c0
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=14045>
<Host h2: h2-eth0:10.0.0.2 pid=14047>
<Host h3: h3-eth0:10.0.0.3 pid=14049>
<Host h4: h4-eth0:10.0.0.4 pid=14051>
<OVSSwitch{'protocols': 'OpenFlow13'} s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None
,s1-eth3:None,s1-eth4:None pid=14056>
<RemoteController{'ip': '10.224.78.132', 'port': 6633} c0: 10.224.78.132:6633 pi
d=14039>
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet> py [(h.name, h.IP(), h.MAC()) for h in net.hosts]
[('h1', '10.0.0.1', '00:00:00:00:00:01'), ('h2', '10.0.0.2', '00:00:00:00:00:02'
), ('h3', '10.0.0.3', '00:00:00:00:00:03'), ('h4', '10.0.0.4', '00:00:00:00:00:0
4')]
mininet>
```

The switch reports is_connected: true and OpenFlow13. Ryu's REST switch inventory returns [1], so only the Lab 7 switch is connected.

Controller connection. Actual terminal screenshot over SSH.

```
97060324-da2c-4cc5-ac47-5e2006b159a7
    Manager "ptcp:6632"
    Bridge "s1"
        Controller "ptcp:6654"
        Controller "tcp:10.224.78.132:6633"
            is_connected: true
        fail_mode: secure
        Port "s1-eth1"
            Interface "s1-eth1"
        Port "s1"
            Interface "s1"
                type: internal
        Port "s1-eth4"
            Interface "s1-eth4"
        Port "s1-eth3"
            Interface "s1-eth3"
        Port "s1-eth2"
            Interface "s1-eth2"
    ovs_version: "2.0.2"
["OpenFlow13"]
[1]
```

**3.** Write a Python function called translate_intent(intent_string) that: **[20 points]**

- Takes a natural-language policy string as input — for example: "Block all traffic from h1 to h3"
- Constructs a prompt that instructs the LLM to act as a network policy translator, accept the intent, and return a JSON object matching your schema from step 1 — nothing else
- Makes an API call to your chosen LLM provider
- Parses the JSON response and returns it as a Python dictionary
- Raises a clear exception if the response cannot be parsed as valid JSON matching your schema

Provide the complete function code. Test it with at least three different intent strings and provide screenshots of the JSON output for each.

## Objective 2 – Apply the Translated Intent to the Network

Translation without application is just text processing. This objective closes the loop: take the structured policy from your translator and turn it into actual OpenFlow rules on a live switch.

**1.** Write a Python function called apply_policy(policy_dict) that takes your translated JSON policy dictionary and applies it to the network by constructing and sending the appropriate REST API call to your controller. The function must handle at minimum: **[25 points]**

- Block rules: install a drop flow entry matching the specified source, destination, and protocol
- Allow rules: install a forward flow entry with appropriate match fields
- The function must print a confirmation message showing what rule was installed on which switch, including the flow entry parameters

Provide the complete function code. You do not need to implement rate-limiting or prioritization in this objective — those are addressed in Objective 3.

**2.** Write a main controller function called run_intent_controller() that combines translation and application: **[15 points]**

- Accepts a natural-language intent string from the user via the terminal (input() is fine)
- Calls translate_intent() to get the structured policy
- Displays the translated policy to the user and asks for confirmation before applying it — print the JSON and prompt: "Apply this policy? (yes/no)"
- If confirmed, calls apply_policy() to install the rule
- Prints a final confirmation that the rule is active

The confirmation step is intentional and important. Explain in one sentence why human-in-the-loop confirmation matters before applying AI-generated network policy.

**3.** Test your complete pipeline with the following four intents. For each one, provide: the original intent string, the translated JSON output, your confirmation prompt, and a screenshot verifying the rule is active in the flow table: **[20 points]**

- "Block all traffic from h1 to h3"
- "Allow only ICMP traffic between h2 and h4"
- "Block all traffic on port 80 from any host to h4"
- One intent of your own design that tests an edge case or a more complex policy

## Objective 3 – Handling Ambiguity and Bad Input

Natural language is ambiguous. Users make mistakes. A production intent-based controller must handle both gracefully — not crash, not silently do the wrong thing, and not apply a policy it does not understand. This objective tests your pipeline against the hard cases.

**1.** Test your translate_intent() function with the following ambiguous or problematic inputs. For each one, document what the LLM returned, whether your code handled it correctly, and what happened in the network: **[20 points]**

- "Make the network faster" — vague, no actionable policy
- "Block h5" — references a host that does not exist in your topology
- "Stop h1 from doing bad things" — semantically meaningless in a network context
- "Allow h1 to h2 but block h1 to h2" — contradictory policy

**2.** Based on your findings from step 1, add a validation layer to your pipeline. Write a Python function called validate_policy(policy_dict) that checks the translated policy before it reaches apply_policy(). The validator must reject: **[20 points]**

- Policies that reference hosts not present in the current topology
- Policies with missing required fields according to your schema
- Contradictory rules that conflict with an already-installed policy

If validation fails, the function must return a clear human-readable error message explaining why the policy was rejected, without crashing. Provide the validator code and demonstrate it catching at least two of the problem cases from step 1.

**3.** In one paragraph: what does this objective teach you about the gap between what LLMs are good at and what production network automation requires? Where does human judgment still need to be in the loop, even in an intent-based system? **[10 points]**

## Extra Credit: Objective 4 – Voice Intent Extension

Text input is one interface. Voice is another — and increasingly the preferred one for operators working in data centers or NOCs where typing is impractical. This objective adds a voice input path to your intent controller using the OpenAI Whisper API.

This objective is more advanced than the previous three. Approach it methodically: get audio capture working first, then transcription, then connect it to your existing pipeline.

**1.** Install the required libraries for audio capture and transcription: **[10 points]**

```
pip install openai-whisper sounddevice scipy
```

Write a Python function called capture_voice_intent() that:

- Records audio from the system microphone for a configurable duration (default: 5 seconds)
- Saves the recording as a temporary WAV file
- Sends it to the Whisper API for transcription
- Returns the transcribed text string

Provide the function code and a screenshot of it successfully transcribing a spoken network policy statement.

**2.** Connect your voice capture to your full intent pipeline. The complete flow must be: **[15 points]**

- User speaks a network policy intent into the microphone
- capture_voice_intent() transcribes it
- translate_intent() translates the transcription
- validate_policy() validates the result
- User confirms the translated policy
- apply_policy() installs the rule

Demonstrate the complete voice-to-network pipeline with at least two spoken intents. Provide screenshots of the transcription output and the resulting flow table entries.

**3.** In two to three sentences: what are the practical limitations of voice-driven network control in a production environment? Under what conditions would it be genuinely useful, and under what conditions would it be a liability? **[8 points]**

> **Lab Report Reminder:** Disclose your AI tool use in your lab report. Submit all Python script files with your lab document. Be prepared to demonstrate your intent controller live — the TA or instructor may ask you to enter an intent and show the resulting flow table entry during your submission review.

### Lab 7 Total: 153 points + 25 points Extra Credit